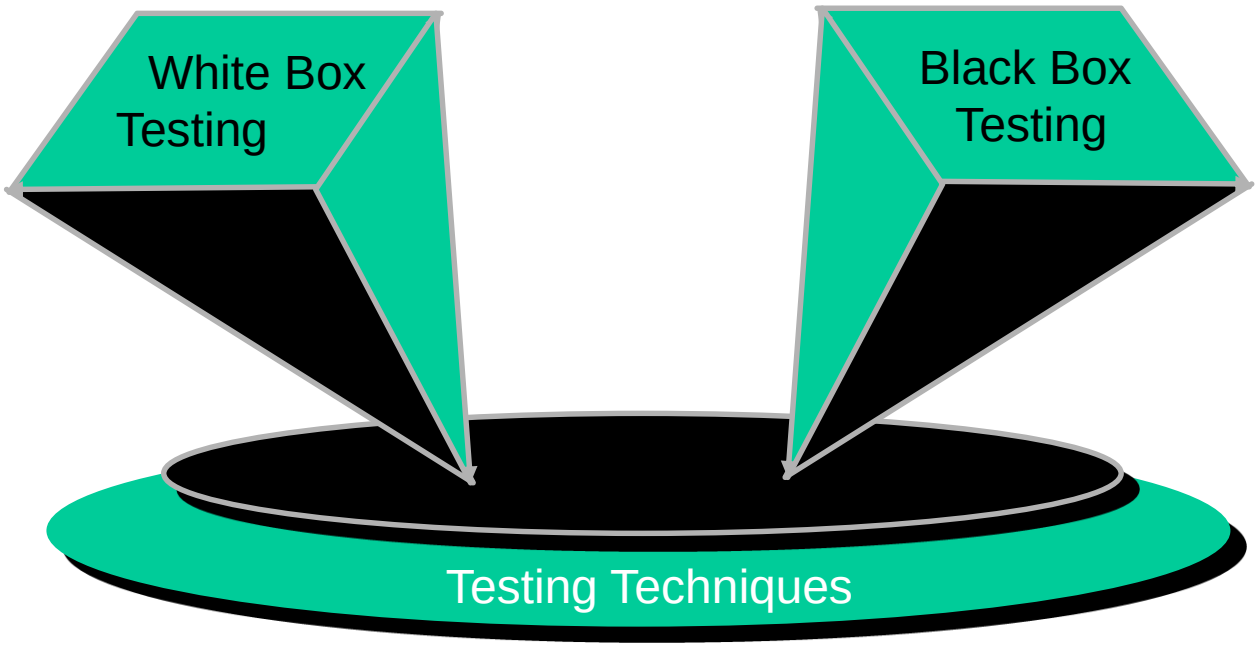
Testing Techniques

Minsoo Ryu

Hanyang University

Two Approaches to Testing



White Box vs. Black Box Testing

- White box testing
  - Knowing the internal workings of a product
  - Can be conducted only when component-level design (or source code) exists

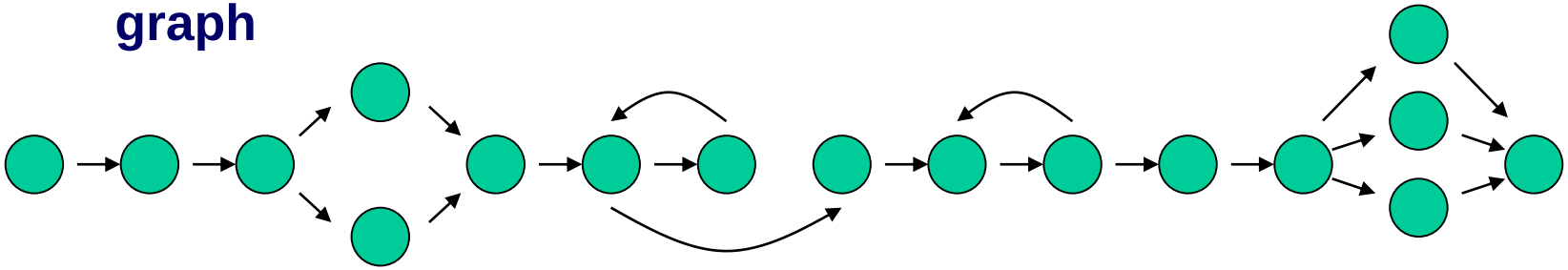
- Black box testing
  - Knowing the specified function that a product has been to perform
  - Alludes to tests that are conducted at the software interface

Topics covered

1. White box testing
   - Basis Path Testing
   - Data Flow Testing
   - State-based Testing
   - Loop Testing
2. Black box testing
   - Equivalence Partitioning
   - Boundary Testing
   - Decision Table Testing
   - Orthogonal Array

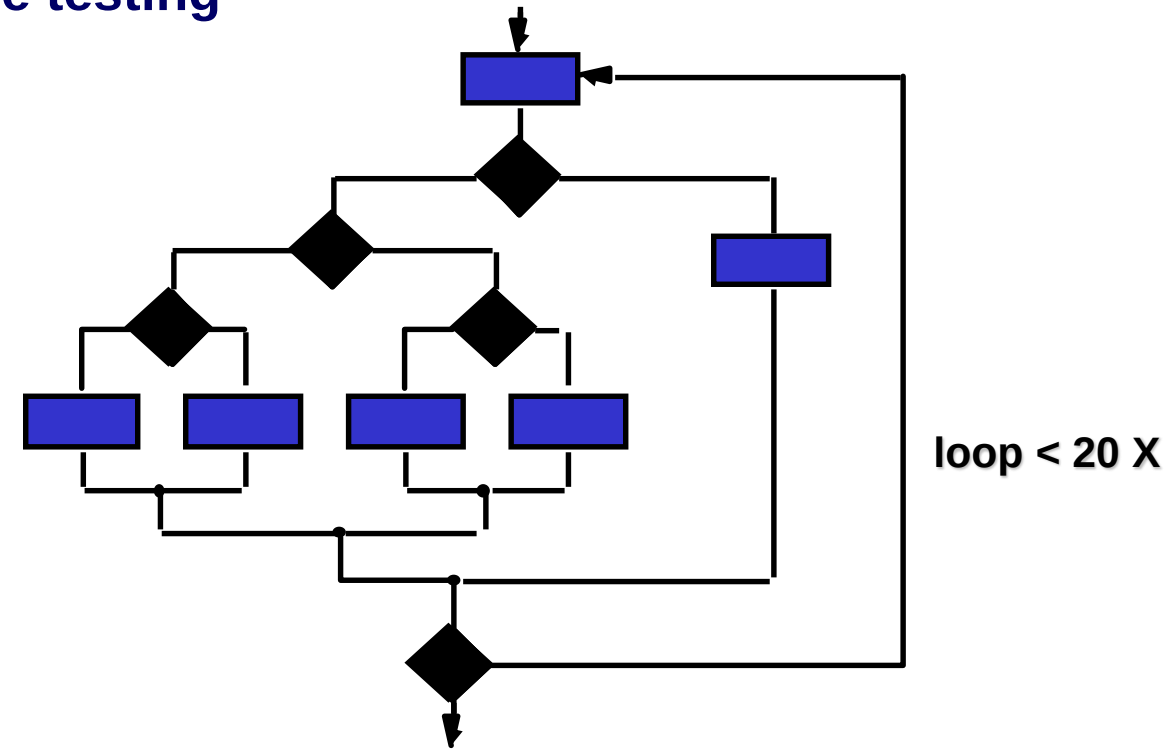
Basis Path Testing

- Basis path testing is a white box testing proposed by Tom McCabe
  - The objective of path testing is to ensure that the set of test cases is such that each path through the program is executed at least once
  - The starting point for path testing is a program flow graph that shows nodes representing program decisions and arcs representing the flow of control
  - Statements with conditions are therefore nodes in the flow graph



Basis Path Testing

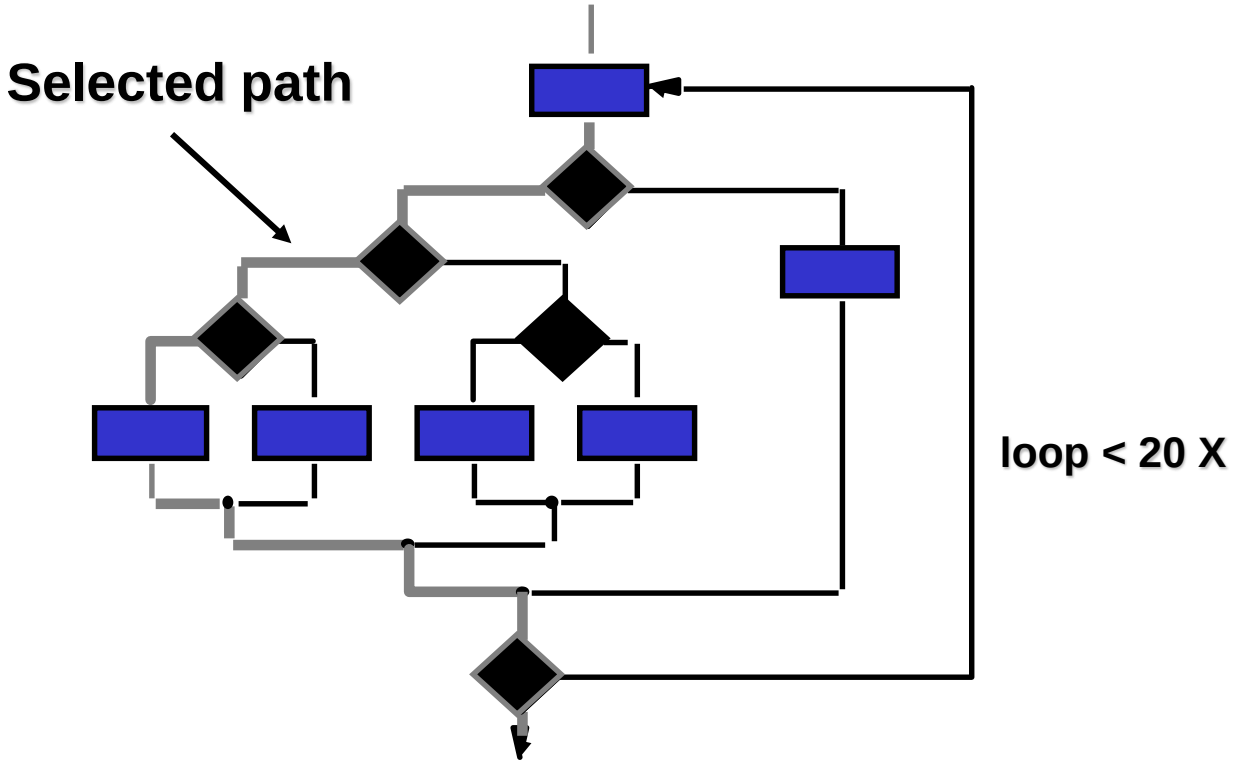
- Exhaustive testing



- There are 5^{20} ($= 10^{14}$) paths for the above small program!
- If the processor can exercise each test case in one millisecond, working 24 hours a day, 365 days a year, the processor would work for 3170 years

Basis Path Testing

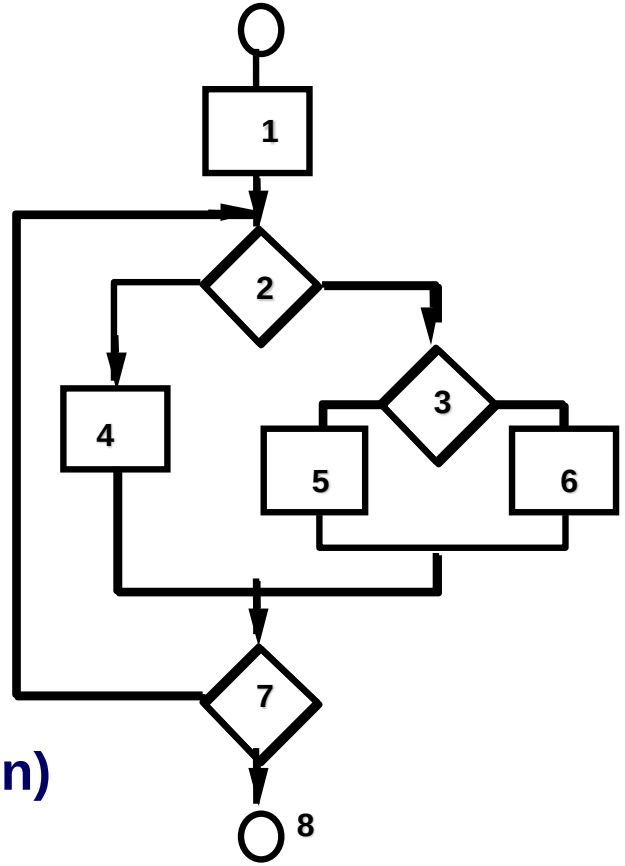
❒ Selective testing



Independent Program Path

- An independent path is any path through the program that introduces at least one new set of processing statements or a new condition
  - An independent path must move along at least one edge that has not been traversed before the path is defined

Path 1: 1,2,3,6,7,8
Path 2: 1,2,3,5,7,8
Path 3: 1,2,4,7,8
Path 4: 1,2,4,7,2,4,...7,8



  - Path <1, 2, 3, 5, 7, 2, 4, 7, 8> is not an independent path (simply a combination)

Independent Program Path

- **Paths 1, 2, 3, 4 constitute a basis set**
  - **The basis set is not unique**
  - **A number of different basis sets can be derived for a given design**

- **How do we know how many paths to look for?**
  - **The computation of cyclomatic complexity provides the answer**
  - **Cyclomatic complexity V(G)**
    - The number of regions
    - The number of edges – the number of nodes + 2
    - The number of predicate nodes + 1
      - ✓ Each node that contains a condition is called a predicate node

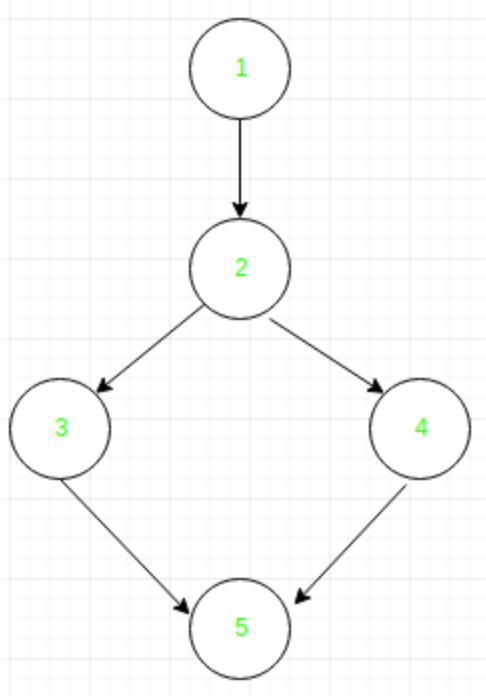
Data Flow Testing

- Selects test paths of a program according to the locations of definitions and uses of variables in the program

- For a statement with S as its statement number
  - DEF(S) = {X | statement S contains a definition of X}
  - USE(S) = {X | statement S contains a use of X}
  - The definition of variable X at statement S is said to be live at statement S' if there exists a path from statement S to statement S' that contains no other definition of X
  - A definition-use (DU) chain of variable X is of the form [X, S, S']
  - One simple data flow testing is to require that every DU chain be covered at least once

Example

```
1. read x, y;
2. if(x>y)
3.     a = x+1
     else
4.     a = y-1
5. print a;
```



Variable	Defined	Used
x	1	2, 3
y	1	2, 4
a	3, 4	5

State-based Testing

- State-based testing defines a set of abstract states that a software unit (object) can take and tests the unit's behavior by comparing its actual states to the expected states
  - This approach is popular with object-oriented systems

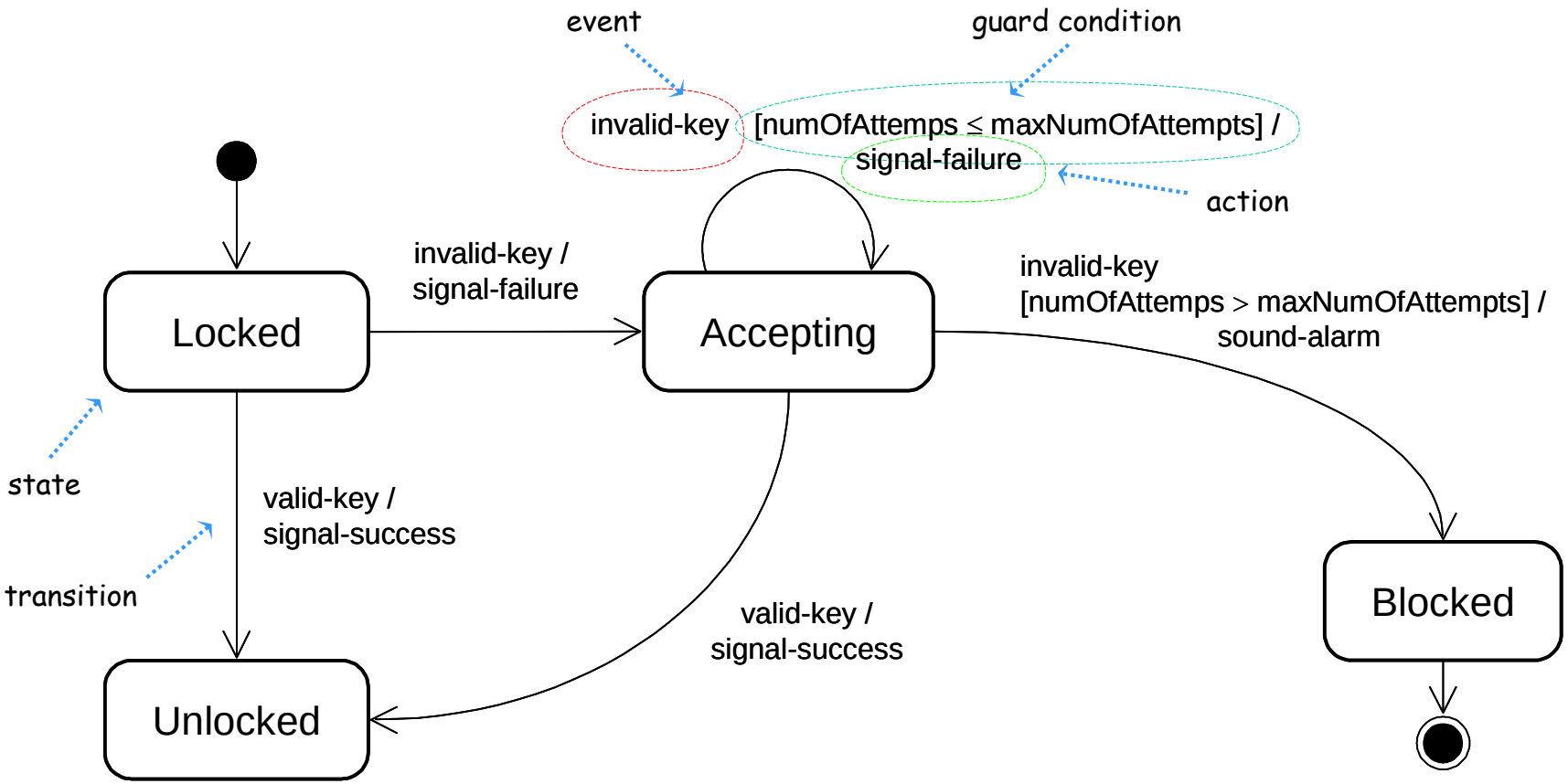
- The state of an object is defined as a constraint on the values of its attributes
  - Because the methods use the attributes in computing the object's behavior, the behavior depends on the object state
  - An object without attributes does not have states, but still can be unit-tested (shown later)

Example

- Define the relevant states as combinations of object attribute values:
  - "Locked" ≡ defined as:
    (lockDeviceArmed == true) && (numOfAttempts == 0)
  - "Accepting" ≡ defined as:
    (lockDeviceArmed == true) && (0 < numOfAttempts < maxNumOfAttempts)
  - "Unlocked" ≡ defined as:
    (lockDeviceArmed == false) && (numOfAttempts == 0)
  - "Blocked" ≡ defined as:
    (lockDeviceArmed == true) && (numOfAttempts == maxNumOfAttempts)
  - "Undefined" ≡ defined as:
    ((numOfAttempts < 0) || (maxNumOfAttempts < numOfAttempts)) ||
    ((lockDeviceArmed == false) && (numOfAttempts != 0))

- Define the relevant events:
  - User entered a valid key
  - User entered an invalid key

Example

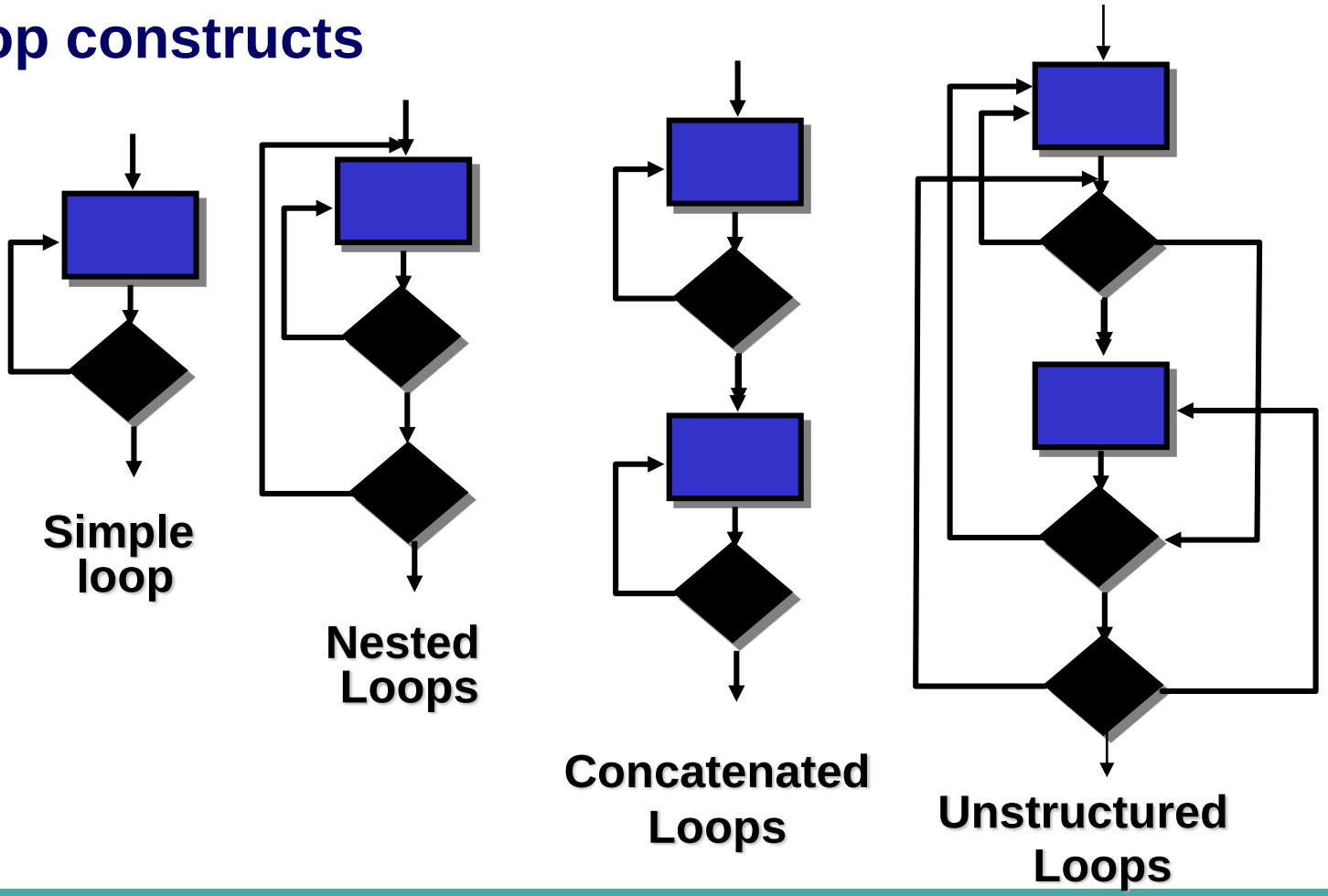

event
guard condition
invalid-key
[numOfAttemps ≤ maxNumOfAttempts] /
signal-failure
action
Locked
invalid-key /
signal-failure
Accepting
invalid-key
[numOfAttemps > maxNumOfAttempts] /
sound-alarm
state
valid-key /
signal-success
transition
valid-key /
signal-success
Blocked
Unlocked

Coverage Conditions

- Cover all identified states at least once
  - Each state is part of at least one test case
- Cover all valid transitions at least once
- Trigger all invalid transitions at least once

Loop Testing

- Loop testing focuses exclusively on the validity of loop constructs



Loop Testing

Simple Loops

1. skip the loop entirely
2. only one pass through the loop
3. two passes through the loop
4. m passes through the loop m < n
5. (n-1), n, and (n+1) passes through the loop

where n is the maximum number of allowable passes

Loop Testing

Nested Loops

Start at the innermost loop. Set all outer loops to their minimum iteration parameter values.

Test the min+1, typical, max-1 and max for the innermost loop, while holding the outer loops at their minimum values.

Move out one loop and set it up as in step 2, holding all other loops at typical values. Continue this step until the outermost loop has been tested.

Concatenated Loops

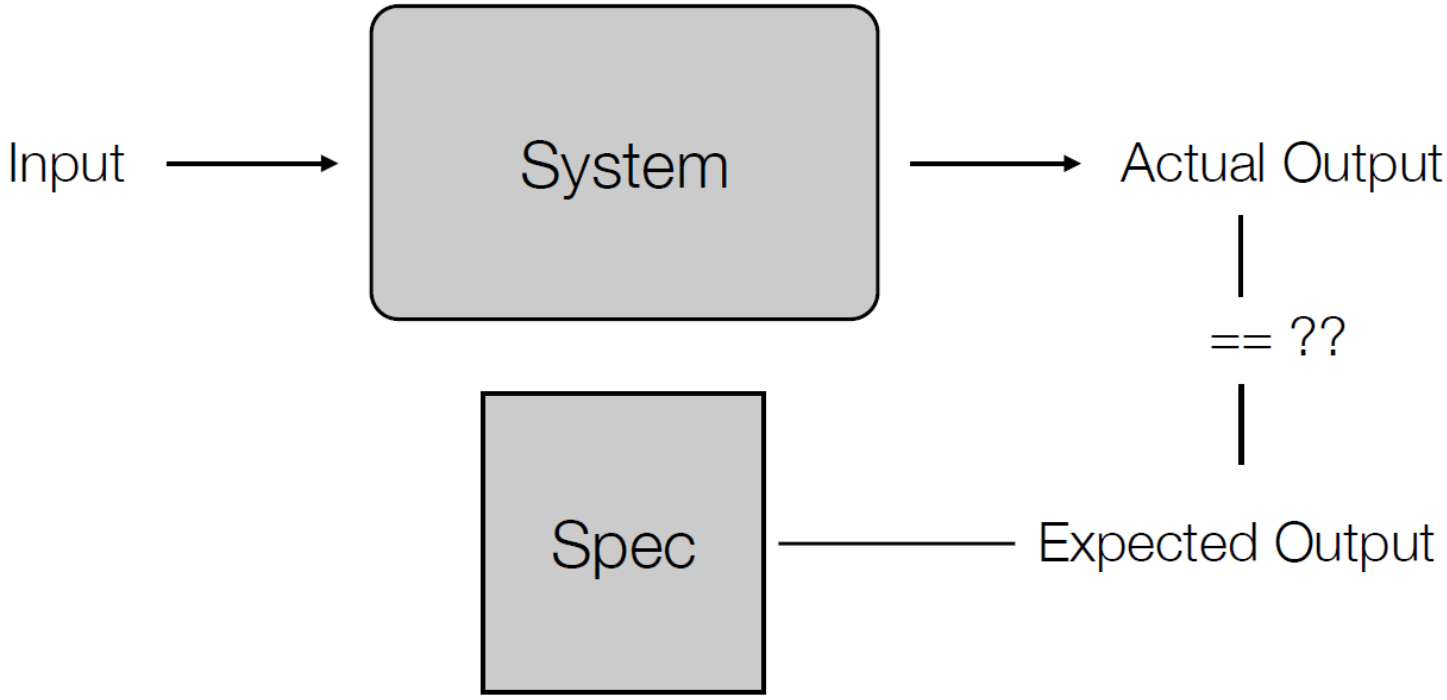
If the loops are independent of one another
then treat each as a simple loop
else* treat as nested loops
endif*

for example, the final loop counter value of loop 1 is used to initialize loop 2.

Topics covered

1. White box testing
   - Basis Path Testing
   - Data Flow Testing
   - State-based Testing
   - Loop Testing
2. Black box testing
   - Equivalence Partitioning
   - Boundary Testing
   - Decision Table Testing
   - Orthogonal Array

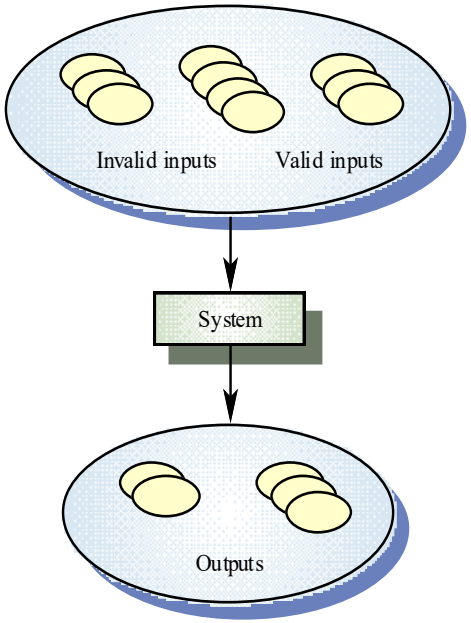
Black Box Testing

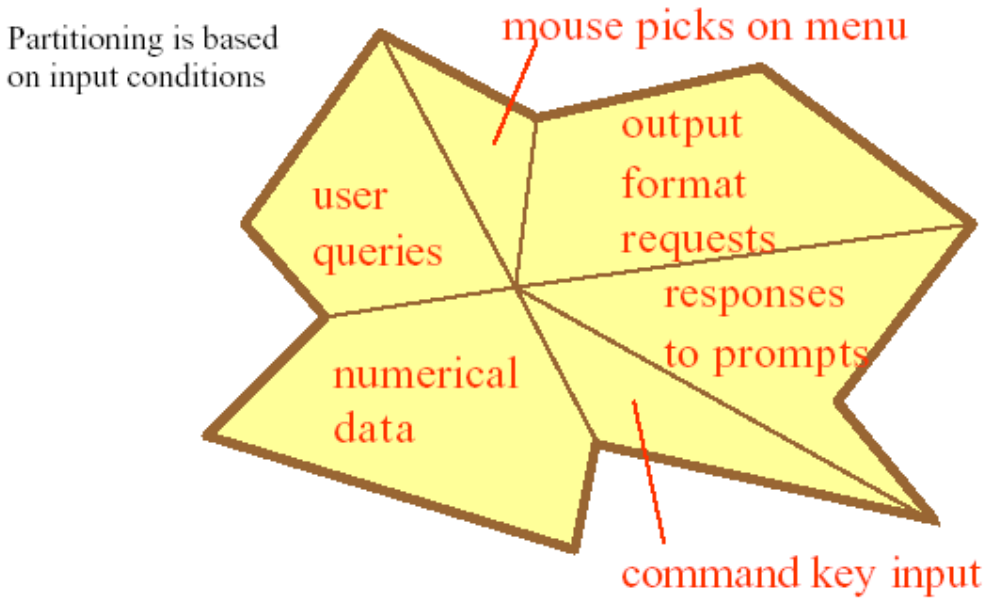


A **black box test** passes **input** to a system, records the **actual output** and compares it to the **expected output**

Equivalence Partitioning

- Equivalence partitioning **divides the input domain** of a program into classes of data from which test cases can be derived
  - Fold the space into **equivalent behaviors** and then sample each partition





Example

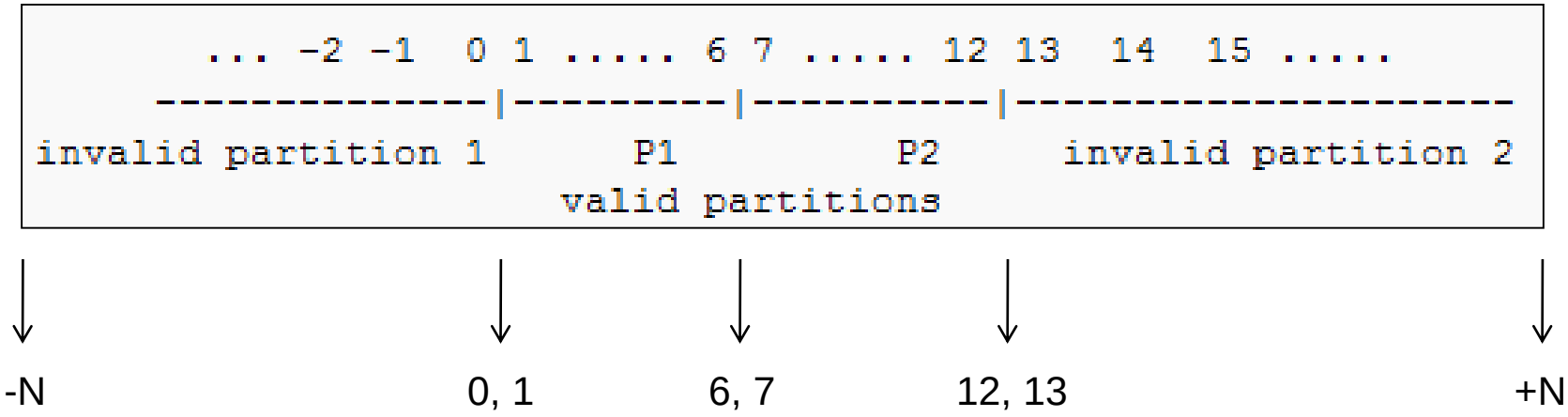
- Consider a simple example like the greatest common denominator function
  - int gcd(int x, int y)
- At first glance, this function has an infinite number of test cases
- But lets fold the space
  - x=6 y=9, returns 3, tests common case
  - x=2 y=4, returns 2, tests when x is the GCD
  - x=3 y=5, returns 1, tests two primes
  - x=9 y=0, returns ?, tests zero
  - x=-3 y=9, returns ?, tests negative

Boundary Testing

- Boundary testing is a special case of equivalence testing that focuses on the boundary values of input parameters
  - Based on the assumption that developers often overlook special cases at the boundary of equivalence classes
- Selects elements from the "edges" of each equivalence class, or "outliers" such as
  - zero, min/max values, empty set, empty string, and null
  - confusion between > and >=

Boundary Testing

- Tests are designed to include representatives of boundary values
  - Values on the minimum and maximum edges of an equivalence partition are tested



Decision Table Testing

- A decision table lists causes and effects in a matrix
- Each column represents a unique combination
- Purpose is to structure logic

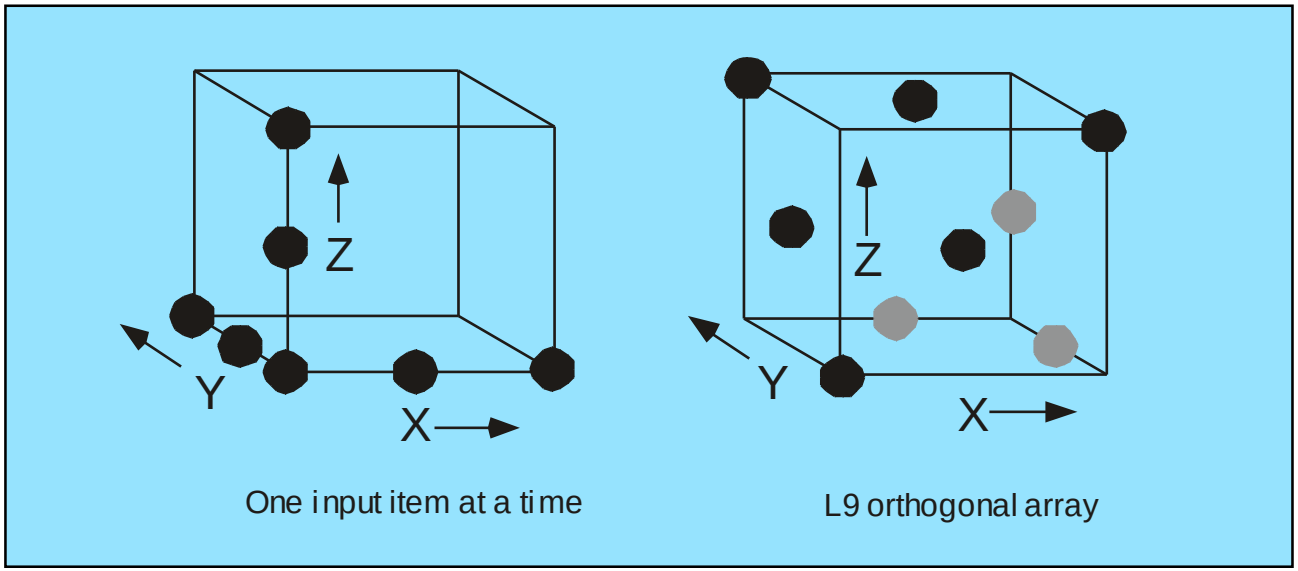
		Combinations							
Causes	**Values**	1	2	3	4	5	6	7	8
Cause 1	*Y, N*	Y	Y	Y	Y	N	N	N	N
Cause 2	*Y, N*	Y	Y	N	N	Y	Y	N	N
Cause 3	*Y, N*	Y	N	Y	N	Y	N	Y	N
Effects									
Effect 1		X			X				X
Effect 2			X				X		X

Example

c1: a<b+c?	F	T	T	T	T	T	T	T	T	T	T
c2: b<a+c?	-	F	T	T	T	T	T	T	T	T	T
c3: c<a+b?	-	-	F	T	T	T	T	T	T	T	T
c4: a = b?	-	-	-	T	T	T	T	F	F	F	F
c5: a = c?	-	-	-	T	T	F	F	T	T	F	F
c6: b = c?	-	-	-	T	F	T	F	T	F	T	F
Rule Count	32	16	8	1	1	1	1	1	1	1	1
a1: Not a Triangle	X	X	X								
a2: Scalene											X
a3: Isosceles							X		X	X	
a4: Equilateral				X							
a5: Impossible					X	X		X			

Orthogonal Array Testing

- Orthogonal array testing can be applied to problems in which the input domain is relatively small but too large to accommodate exhaustive testing
  - In any two of columns, each pair of symbols occurs exactly the same number of times



Orthogonal Array Testing

- P1 = {0, 1, 2}, P2 = {0, 1, 2}, P3 = {0, 1, 2}, P4 = {0, 1, 2}

P1	P2	P3	P4
0	0	0	0
1	0	0	0
2	0	0	0
0	1	0	0
0	2	0	0
0	0	1	0
0	0	2	0
0	0	0	1
0	0	0	2

<One at a time>

P1	P2	P3	P4
0	0	2	1
0	1	0	0
0	2	1	2
2	0	0	2
2	1	1	1
2	2	2	0
1	0	1	0
1	1	2	2
1	2	0	1

<Orthogonal array>

Orthogonal Array

The $L_a(b^c)$ notation can be decomposed as follows:

- L: Symbol for an orthogonal array
- a: The number of experiments required for this particular array
- b: Specifies the number of test levels allowed for each factor
- c: Specifies the number of factors which this array can examine

For example:

The $L_8(2^7)$ specifies that this matrix is an orthogonal array that requires eight experiments. In these eight experiments, seven different two-level factors can be evaluated.

Orthogonal Array

Notation

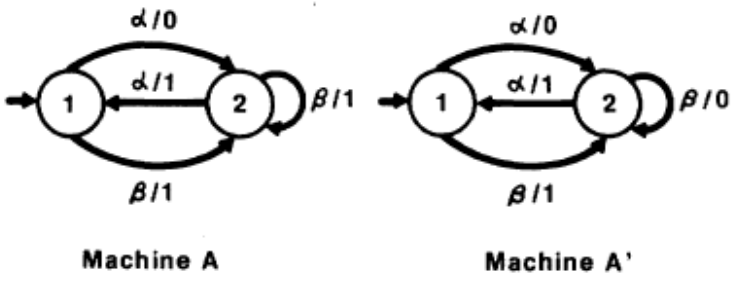
An example of a frequently used orthogonal array is shown in Table 3.1. This particular balanced matrix is termed orthogonal distribution array $L_8(2^7)$, more commonly referred to as simply an L_8 orthogonal array.

Exp # \ Col.	(1)	(2)	(3)	(4)	(5)	(6)	(7)
1	1	1	1	1	1	1	1
2	1	1	1	2	2	2	2
3	1	2	2	1	1	2	2
4	1	2	2	2	2	1	1
5	2	1	2	1	2	1	2
6	2	1	2	2	1	2	1
7	2	2	1	1	2	2	1
8	2	2	1	2	1	1	2

Table 3.1
L_8 Orthogonal Array

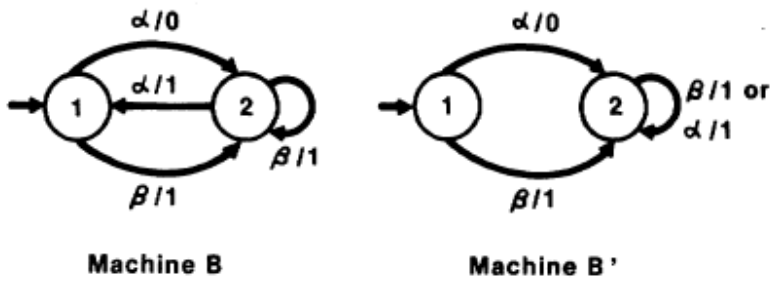

Q & A

Chow's Operation Errors and Transfer Errors



Machine A has an operation error at State 2 with input β. Such operation errors can be detected by a "branch cover" $\alpha\alpha\beta\beta$.

Fig. 5. Operation errors.



Machine B has a transfer error at State 2 with input α. This transfer error cannot be detected by a "branch cover" for A: $\alpha\alpha\beta\beta$.

Fig. 6. Transfer errors.

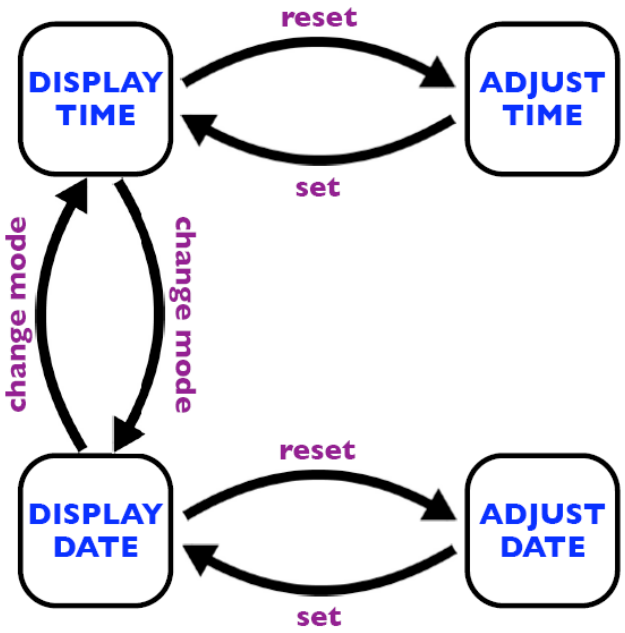
- **Operation error**
  - A is said to have operation errors if A is not equivalent to A' and A can be modified to be equivalent to A' by changing only the output function of A (1 → 0)
  - A' is the standard or the correct version
- **Transfer error**
  - A is said to have transfer errors if A is not equivalent to A' and A can be modified to be equivalent to A' by changing only the next-state function of A

Switch Coverage (N-1 Switch Cover)

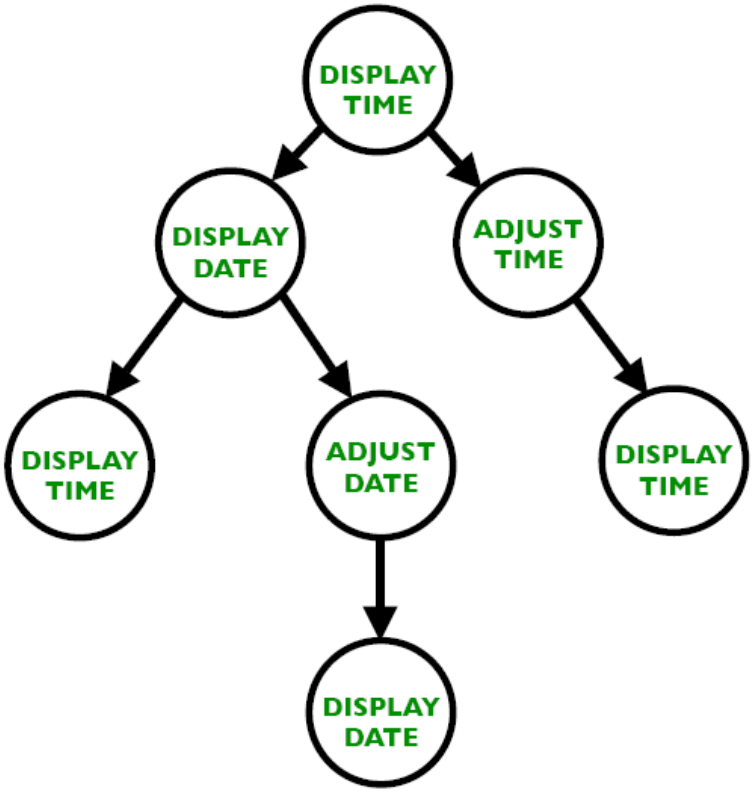
- **0-Switch Cover or Branch Coverage**
  - **Every branch (state transition edge) is traversed**

- **1-Switch Cover or Switch Coverage**
  - **Every branch-to-branch is traversed**

0-Switch Cover or Branch Coverage



<State Diagram>



<0-switch or Branch Coverage>

Test Cases

❒ 0-Switch Cover

TEST 1

	STEP 1	STEP 2
START STATE	DISPLAY TIME	DISPLAY DATE
INPUT	CHANGE MODE	CHANGE MODE
OUTPUT	DISPLAY DATE	DISPLAY TIME
FINISH STATE	DISPLAY DATE	DISPLAY TIME

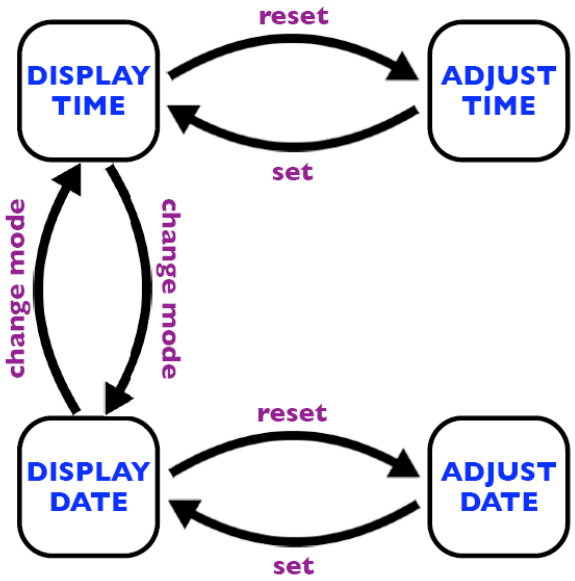
TEST 2

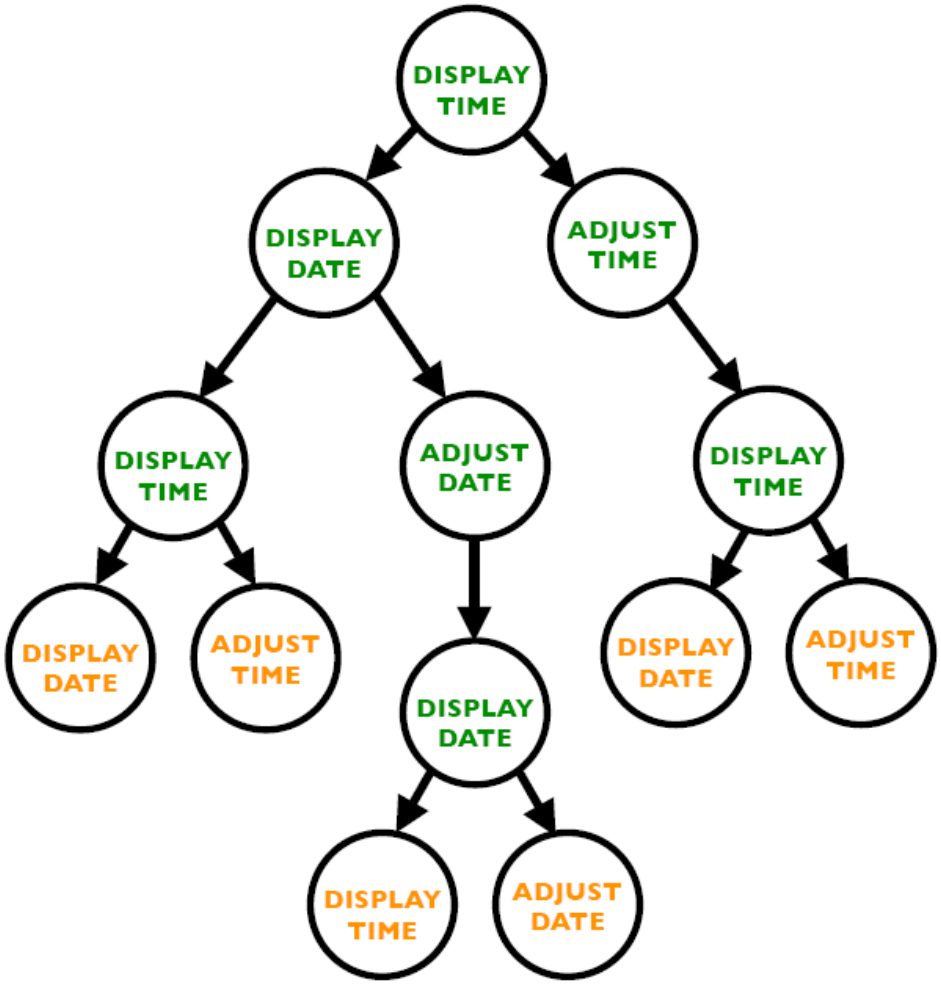
	STEP 1	STEP 2	STEP 3
START STATE	DISPLAY TIME	DISPLAY DATE	ADJUST DATE
INPUT	CHANGE MODE	RESET	SET
OUTPUT	DISPLAY DATE	ADJUST DATE	DISPLAY DATE
FINISH STATE	DISPLAY DATE	ADJUST DATE	DISPLAY DATE

TEST 3

	STEP 1	STEP 2
START STATE	DISPLAY TIME	ADJUST TIME
INPUT	RESET	SET
OUTPUT	ADJUST TIME	DISPLAY TIME
FINISH STATE	ADJUST TIME	DISPLAY TIME

1-Switch Cover or Switch Coverage





Test Cases

TEST 1

	STEP 1	STEP 2	STEP 3
START STATE	DISPLAY TIME	DISPLAY DATE	DISPLAY TIME
INPUT	CHANGE MODE	CHANGE MODE	CHANGE MODE
OUTPUT	DISPLAY DATE	DISPLAY TIME	DISPLAY DATE
FINISH STATE	DISPLAY DATE	DISPLAY TIME	DISPLAY DATE

TEST 2

	STEP 1	STEP 2	STEP 3
START STATE	DISPLAY TIME	DISPLAY DATE	DISPLAY TIME
INPUT	CHANGE MODE	CHANGE MODE	RESET
OUTPUT	DISPLAY DATE	DISPLAY TIME	ADJUST TIME
FINISH STATE	DISPLAY DATE	DISPLAY TIME	ADJUST TIME

TEST 3

	STEP 1	STEP 2	STEP 3	STEP 4
START STATE	DISPLAY TIME	DISPLAY DATE	ADJUST DATE	DISPLAY DATE
INPUT	CHANGE MODE	RESET	SET	CHANGE MODE
OUTPUT	DISPLAY DATE	ADJUST DATE	DISPLAY DATE	DISPLAY TIME
FINISH STATE	DISPLAY DATE	ADJUST DATE	DISPLAY DATE	DISPLAY TIME

TEST 4

	STEP 1	STEP 2	STEP 3	STEP 4
START STATE	DISPLAY TIME	DISPLAY DATE	ADJUST DATE	DISPLAY DATE
INPUT	CHANGE MODE	RESET	SET	RESET
OUTPUT	DISPLAY DATE	ADJUST DATE	DISPLAY DATE	ADJUST DATE
FINISH STATE	DISPLAY DATE	ADJUST DATE	DISPLAY DATE	ADJUST DATE

TEST 5

	STEP 1	STEP 2	STEP 3
START STATE	DISPLAY TIME	ADJUST TIME	DISPLAY TIME
INPUT	RESET	SET	CHANGE MODE
OUTPUT	ADJUST TIME	DISPLAY TIME	DISPLAY DATE
FINISH STATE	ADJUST TIME	DISPLAY TIME	DISPLAY DATE

TEST 6

	STEP 1	STEP 2	STEP 3
START STATE	DISPLAY TIME	ADJUST TIME	DISPLAY TIME
INPUT	RESET	SET	RESET
OUTPUT	ADJUST TIME	DISPLAY TIME	ADJUST TIME
FINISH STATE	ADJUST TIME	DISPLAY TIME	ADJUST TIME